1️⃣ Overview of SQL

What is SQL?

SQL (Structured Query Language) is a declarative language used to:

- Define database structures
- Manipulate data
- Retrieve data
- Control access

It works with **Relational Database Management Systems (RDBMS)** like:

- Microsoft SQL Server
- MySQL
- Oracle
- PostgreSQL

SQL is Declarative

Unlike C# or Java (procedural), SQL is declarative.

You don't say **how** to fetch data.
You say **what** data you want.

Example:

```
SELECT * FROM Employees;
```

You are not telling SQL Server how to scan the table — the query optimizer decides that.

SQL Execution Order (Logical Processing Order)

Very important for interviews:

1. FROM
2. JOIN

3. WHERE
4. GROUP BY
5. HAVING
6. SELECT
7. ORDER BY

2️⃣ SQL Command Types

SQL commands are categorized as:

Type	Meaning	Purpose
DDL	Data Definition Language	Structure
DML	Data Manipulation Language	Data changes
DQL	Data Query Language	Retrieve data
DCL	Data Control Language	Permissions
TCL	Transaction Control Language	Transactions

🔹 DDL (Data Definition Language)

Used to define or modify database structure.

Commands:

- CREATE
- ALTER
- DROP
- TRUNCATE

🔹 DML (Data Manipulation Language)

Used to manipulate data inside tables.

Commands:

- INSERT
- UPDATE
- DELETE

🔹 **DQL (Data Query Language)**

- SELECT

🔹 **DCL**

- GRANT
- REVOKE

🔹 **TCL**

- COMMIT
- ROLLBACK
- SAVEPOINT

3️⃣ DDL Commands (In Detail)

🔹 **CREATE**

Creates database objects.

```
CREATE TABLE Employees (
    EmployeeId INT,
    Name VARCHAR(100),
    Salary DECIMAL(10,2)
```

);

🔹 ALTER

Modify existing structure.

ALTER TABLE Employees

ADD Email VARCHAR(150);

🔹 DROP

Deletes table permanently.

DROP TABLE Employees;

🔹 TRUNCATE

Removes all records (faster than DELETE).

TRUNCATE TABLE Employees;

Difference:

DELETE	TRUNCATE
Can use WHERE	Cannot use WHERE
Slower	Faster
Logged row-by-row	Minimal logging

4️⃣ Constraints

Constraints enforce rules at the database level.

🔹 PRIMARY KEY

- Uniquely identifies a record

- Cannot be NULL
- Only one per table

```
CREATE TABLE Students (
   StudentId INT PRIMARY KEY,
   Name VARCHAR(100)
);
```

🔹 FOREIGN KEY

Creates relationship between tables.

```
CREATE TABLE Orders (
   OrderId INT PRIMARY KEY,
   StudentId INT,
   FOREIGN KEY (StudentId) REFERENCES Students(StudentId)
);
```

🔹 UNIQUE

No duplicate values allowed.

```
Email VARCHAR(100) UNIQUE
```

🔹 NOT NULL

Column must have value.

🔹 CHECK

Validates condition.

```
Age INT CHECK (Age >= 18)
```

🔹 **DEFAULT**

Sets default value.

Country VARCHAR(50) DEFAULT 'India'

5️⃣ Querying Data (SELECT)

Basic syntax:

```
SELECT column1, column2
FROM table_name;
```

Example:

```
SELECT Name, Salary
FROM Employees;
```

SELECT All Columns

```
SELECT * FROM Employees;
```

SELECT with Expressions

```
SELECT Name, Salary * 12 AS AnnualSalary
FROM Employees;
```

6️⃣ Sorting Data (ORDER BY)

Used to sort results.

```
SELECT *
FROM Employees
ORDER BY Salary ASC;
```

Descending:

```
ORDER BY Salary DESC;
```

Multiple columns:

ORDER BY DepartmentId, Salary DESC;

7️⃣ Filtering Data (WHERE)

Filters rows.

```
SELECT *
FROM Employees
WHERE Salary > 50000;
```

Operators

Operator	Meaning
=	Equal
<>	Not equal
>	Greater than
<	Less than
BETWEEN	Range
IN	Multiple values
LIKE	Pattern matching

Example:

```
SELECT *
FROM Employees
WHERE Name LIKE 'S%';
```

8️⃣ Joining Tables

Joins combine data from multiple tables.

Assume:

Employees

| EmployeeId | Name | DepartmentId |

Departments

| DepartmentId | DepartmentName |

🔹 INNER JOIN

Returns matching records.

```
SELECT e.Name, d.DepartmentName
FROM Employees e
INNER JOIN Departments d
ON e.DepartmentId = d.DepartmentId;
```

🔹 LEFT JOIN

Returns all left table records.

```
SELECT e.Name, d.DepartmentName
FROM Employees e
LEFT JOIN Departments d
ON e.DepartmentId = d.DepartmentId;
```

🔹 RIGHT JOIN

Returns all right table records.

🔹 **FULL JOIN**

Returns all records from both tables.

9️⃣ Grouping Data (GROUP BY)

Used with aggregate functions.

Aggregate Functions

- COUNT()
- SUM()
- AVG()
- MIN()
- MAX()

Example:

```
SELECT DepartmentId, COUNT(*) AS TotalEmployees
FROM Employees
GROUP BY DepartmentId;
```

GROUP BY with HAVING

HAVING filters grouped data.

```
SELECT DepartmentId, COUNT(*) AS TotalEmployees
FROM Employees
GROUP BY DepartmentId
HAVING COUNT(*) > 5;
```

🔟 Important Concept: WHERE vs HAVING

WHERE	HAVING
Filters rows	Filters groups
Used before GROUP BY	Used after GROUP BY
Cannot use aggregates	Can use aggregates